

Data Mining & Machine Learning

Exam Summary

Questions, Definitions, Algorithms & Formulas

Complete Exam Revision Notes

Academic Year 2025–2026

2 – Know Your Data

Central Tendency

$$\bar{x} = \frac{1}{N} \sum_{i=1}^N x_i \quad \bar{x}_w = \frac{\sum_{i=1}^N w_i x_i}{\sum_{i=1}^N w_i}$$

$$\text{Med} = L_1 + \left(\frac{N/2 - (\sum \text{freq})_l}{\text{freq}_{\text{median}}} \right) \text{width}$$

$$\text{mean} - \text{mode} = 3(\text{mean} - \text{median})$$

$$\text{mode} = 3 \text{median} - 2 \text{mean}$$

Trimmed mean: remove extreme values, then compute the mean.

Dispersion

$$\sigma^2 = \frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})^2 = \frac{1}{N} \sum_{i=1}^N x_i^2 - \bar{x}^2$$

$$\sigma = \sqrt{\sigma^2} \quad IQR = Q_3 - Q_1$$

Nominal & Binary Dissimilarity

$$d_{\text{match}}(i, j) = \frac{p - m}{p}$$

	$j = 1$	$j = 0$
$i = 1$	q	r
$i = 0$	s	t

$$d_{\text{sym}}(i, j) = \frac{r + s}{q + r + s + t}$$

$$d_{\text{asym}}(i, j) = \frac{r + s}{q + r + s}$$

$$J(i, j) = \text{coherence}(i, j) = \frac{q}{q + r + s}$$

$$d_{\text{asym}}(i, j) = 1 - J(i, j)$$

Standardization

$$z = \frac{x - \mu}{\sigma}$$

$$s_f = \frac{1}{N} \sum_{i=1}^N |x_{if} - m_f| \quad z_{if} = \frac{x_{if} - m_f}{s_f}$$

Minkowski Distances

$$d_h(i, j) = \left(\sum_{k=1}^p |x_{ik} - x_{jk}|^h \right)^{1/h}$$

$$L_1 : d(i, j) = \sum_{k=1}^p |x_{ik} - x_{jk}|$$

$$L_2 : d(i, j) = \sqrt{\sum_{k=1}^p |x_{ik} - x_{jk}|^2}$$

$$L_\infty : d(i, j) = \max_{1 \leq k \leq p} |x_{ik} - x_{jk}|$$

$$d_H(i, j) = \#\{k : x_{ik} \neq x_{jk}\}$$

Ordinal Attributes

$$r_{if} \in \{1, \dots, M_f\} \quad z_{if} = \frac{r_{if} - 1}{M_f - 1}$$

Mixed Attributes

$$d(i, j) = \frac{\sum_{f=1}^p w_{ij}^{(f)} d_{ij}^{(f)}}{\sum_{f=1}^p w_{ij}^{(f)}}$$

$$d_{ij}^{(f)} = \begin{cases} 0 & x_{if} = x_{jf} \\ 1 & x_{if} \neq x_{jf} \end{cases} \quad (\text{binary/nominal})$$

Cosine Similarity

$$\cos(d_1, d_2) = \frac{d_1 \cdot d_2}{\|d_1\| \|d_2\|}$$

$$d_1 \cdot d_2 = \sum_{k=1}^p d_{1k} d_{2k} \quad \|d\| = \sqrt{\sum_{k=1}^p d_k^2}$$

3 – Preprocessing

Rectangular Average Smooth Filter

For a 3-point filter:

$$S_j = \frac{Y_{j-1} + Y_j + Y_{j+1}}{3}$$

More generally:

$$S_i = \frac{1}{m} \sum_{k=-h}^h Y_{i+k}, \quad m = 2h + 1$$

Larger window $\Rightarrow$ more noise reduction, but greater loss of sharp features/peaks.

Savitzky–Golay Smoothing Filter

General form:

$$S_i = \sum_{n=-n_L}^{n_R} c_n Y_{i+n}$$

$$n_L = \# \text{ points left}, \quad n_R = \# \text{ points right}$$

Local polynomial fitted by least squares.

Filter notation:

$$(n_L, n_R, \text{degree})$$

Example:

$$(16, 16, 0)$$

Degree 0 $\Rightarrow$ rectangular-type filter. Higher degree $\Rightarrow$ better peak preservation.

Smoothing Ratio

$$\text{Smoothing Ratio} = \frac{\text{smooth width}}{\# \text{ points in peak half-width}}$$

Higher ratio:

$\Rightarrow$ better SNR, lower peak amplitude, larger bandwidth

Peak height/width preservation:

$$\text{Smoothing Ratio} < 0.2$$

Chi-Square Test χ^2

For **nominal/categorical** variables:

$$\chi^2 = \sum_{i,j} \frac{(o_{ij} - e_{ij})^2}{e_{ij}}$$

o_{ij} = observed frequency

e_{ij} = expected frequency under independence

$$e_{ij} = \frac{(\text{row total}_i)(\text{column total}_j)}{N}$$

Equivalently:

$$e_{ij} = N P(A = a_i) P(B = b_j)$$

$$\chi^2 \approx 0 \Rightarrow \text{independence}$$

$$\chi^2 \text{ large} \Rightarrow \text{association}$$

Degrees of Freedom

For an $r \times c$ contingency table:

$$df = (r - 1)(c - 1)$$

Decision rule:

$$p < \alpha \Rightarrow \text{reject } H_0$$

or equivalently:

$$\chi_{\text{obs}}^2 \geq \chi_{\text{critical}}^2 \Rightarrow \text{reject } H_0$$

ChiMerge

Supervised, bottom-up discretization.

$$\chi^2(I_i, I_{i+1}) \text{ small} \Rightarrow \text{merge}$$

$$\chi^2(I_i, I_{i+1}) \text{ large} \Rightarrow \text{keep separate}$$

Repeatedly merge the adjacent pair with minimum χ^2 until:

$$\chi^2 > \text{threshold}$$

for every adjacent pair, or the desired number of intervals is reached.

Pearson Correlation

For numerical attributes A, B :

$$r_{A,B} = \frac{\sum_i (a_i - \bar{A})(b_i - \bar{B})}{n \sigma_A \sigma_B}$$

$$-1 \leq r_{A,B} \leq 1$$

$$r > 0 \Rightarrow \text{positive correlation}$$

$$r < 0 \Rightarrow \text{negative correlation}$$

$$r = 0 \Rightarrow \text{uncorrelated}$$

Pearson measures **linear** relationships.

Standardization:

$$a'_k = \frac{a_k - \text{mean}(A)}{\text{std}(A)}$$

$$b'_k = \frac{b_k - \text{mean}(B)}{\text{std}(B)}$$

Standardized dot-product view:

$$\text{correlation}(A, B) = A' \cdot B'$$

Covariance

$$\text{Cov}(A, B) = E[(A - \bar{A})(B - \bar{B})]$$

$$\text{Cov}(A, B) = \frac{1}{n} \sum_i a_i b_i - \bar{A} \bar{B}$$

Relation with Pearson:

$$r_{A,B} = \frac{\text{Cov}(A, B)}{\sigma_A \sigma_B}$$

Mutual Information

For discrete variables X, Y :

$$I(X, Y) = \sum_{x \in X} \sum_{y \in Y} p(x, y) \log \frac{p(x, y)}{p(x)p(y)}$$

Independence:

$$p(x, y) = p(x)p(y) \Rightarrow I(X, Y) = 0$$

Higher $I(X, Y) \Rightarrow$ stronger dependency.

Entropy

$$H(D) = - \sum_i p_i \log_2 p_i$$

Single class:

$$H(D) = 0$$

Entropy is larger as the distribution becomes more uniform.

NMIFS

Normalized Mutual Information:

$$NI(f_i, f_s) = \frac{I(f_i; f_s)}{\min(H(f_i), H(f_s))}$$

Feature-selection criterion:

$$G_i = MI(C, f_i) - \frac{1}{|S|} \sum_{f_s \in S} NI(f_i; f_s)$$

Interpretation:

$$\text{Score} = \text{relevance to class} - \text{redundancy}$$

First feature:

$$f^* = \arg \max_{f_i} I(C, f_i)$$

Following features:

$$f^* = \arg \max_{f_i} G_i$$

NMIFS is **supervised**.

Linear Regression

Simple linear regression:

$$y = w_1 x + w_0$$

$$w_1 = \frac{\sum_{i=1}^D (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^D (x_i - \bar{x})^2}$$

$$w_0 = \bar{y} - w_1 \bar{x}$$

Multiple Linear Regression

$$y = w_0 + w_1 x_1 + w_2 x_2 + \dots$$

Matrix solution:

$$W = (X^T X)^{-1} X^T Y$$

Polynomial regression:

$$y = w_0 + w_1 x + w_2 x^2 + w_3 x^3$$

It is nonlinear in x , but **linear in the parameters** w_i .

Sampling

With replacement:

$$\text{Cov}(X_1, X_2) = 0$$

because the draws are independent.

Without replacement:

$$\text{Cov}(X_1, X_2) \neq 0$$

because the draws are dependent.

Normalization

Min-Max:

$$v' = \frac{v - \min_A}{\max_A - \min_A} (\text{new_max}_A - \text{new_min}_A) + \text{new_min}_A$$

Z-score:

$$v' = \frac{v - \mu_A}{\sigma_A}$$

Interpretation:

$$v' = \# \text{ standard deviations from the mean}$$

Decimal Scaling:

$$v' = \frac{v}{10^j}$$

where j is the smallest integer such that:

$$\max(|v'|) < 1$$

4 – Classification

Fayyad–Irani Supervised Discretization

For a continuous attribute A , sort the examples by A . Candidate cut points are only **boundary points**:

$$T = \frac{A(e_i) + A(e_{i+1})}{2}, \quad \text{class}(e_i) \neq \text{class}(e_{i+1})$$

Let $N = |S|$. Class entropy:

$$Ent(S) = - \sum_{i=1}^k P(C_i, S) \log_2 P(C_i, S)$$

Given cut point T :

$$S_1 = \{x \in S : A(x) \leq T\}, \quad S_2 = S - S_1$$

Partition entropy:

$$EP(A, T; S) = \frac{|S_1|}{N} Ent(S_1) + \frac{|S_2|}{N} Ent(S_2)$$

Best cut:

$$T_A = \arg \min_{T \in B_A} EP(A, T; S)$$

Information gain:

$$G(A, T; S) = Ent(S) - EP(A, T; S)$$

equivalently:

$$T_A = \arg \max_{T \in B_A} G(A, T; S)$$

Stopping condition:

$$G(A, T_A; S) > \frac{\log_2(N-1)}{N} + \frac{\Delta(A, T_A; S)}{N}$$

where:

$$\Delta(A, T_A; S) = \log_2(3^c - 2) - [c Ent(S) - c_1 Ent(S_1) - c_2 Ent(S_2)]$$

c, c_1, c_2 : number of represented classes in S, S_1, S_2 .

If satisfied, recursively apply the procedure to S_1 and S_2 .

Decision Tree – Information Gain

For dataset D with m classes:

$$Info(D) = - \sum_{i=1}^m p_i \log_2 p_i, \quad p_i = \frac{|C_i, D|}{|D|}$$

If A generates v partitions:

$$Info_A(D) = \sum_{j=1}^v \frac{|D_j|}{|D|} Info(D_j)$$

Information Gain:

$$Gain(A) = Info(D) - Info_A(D)$$

Select:

$$A^* = \arg \max_A Gain(A)$$

Continuous-Valued Attributes

Sort the values and consider consecutive midpoints:

$$T_i = \frac{a_i + a_{i+1}}{2}$$

For each T_i :

$$D_1 = \{x : A(x) \leq T_i\}, \quad D_2 = \{x : A(x) > T_i\}$$

$$Info_{A, T_i}(D) = \frac{|D_1|}{|D|} Info(D_1) + \frac{|D_2|}{|D|} Info(D_2)$$

Choose:

$$T^* = \arg \min_T Info_{A, T}(D) = \arg \max_T Gain(A, T)$$

Preferably, discretize continuous attributes beforehand (e.g. Fayyad–Irani).

Gain Ratio – C4.5

Information Gain is biased toward attributes with many values.

$$SplitInfo_A(D) = - \sum_{j=1}^v \frac{|D_j|}{|D|} \log_2 \left(\frac{|D_j|}{|D|} \right)$$

$$GainRatio(A) = \frac{Gain(A)}{SplitInfo_A(D)}$$

Select:

$$A^* = \arg \max_A GainRatio(A)$$

$SplitInfo$ measures the fragmentation produced by A , not class impurity.

Gini Index – CART

$$Gini(D) = 1 - \sum_{i=1}^m p_i^2$$

$Gini(D) = 0$ for a pure node.

CART considers **binary splits**:

$$D \rightarrow D_1, D_2$$

Weighted impurity:

$$Gini_A(D) = \frac{|D_1|}{|D|} Gini(D_1) + \frac{|D_2|}{|D|} Gini(D_2)$$

Reduction in impurity:

$$\Delta Gini(A) = Gini(D) - Gini_A(D)$$

Select:

$$A^* = \arg \max_A \Delta Gini(A)$$

equivalently:

$$A^* = \arg \min_A Gini_A(D)$$

For a categorical attribute with $v > 2$ values, test possible binary groupings of its values.

DT Basic Stopping Conditions

Stop if all tuples in D belong to the same class:

$$D \subseteq C_i \Rightarrow \text{leaf} = C_i$$

If no attributes remain:

$$\text{attribute_list} = \emptyset$$

use majority voting:

$$\text{leaf} = \arg \max_{C_i} |C_{i,D}|$$

If a generated partition is empty:

$$D_j = \emptyset$$

attach a leaf labeled with the majority class of the parent set D .

Split Selection Summary

$IG : \max Gain(A)$ $GR : \max GainRatio(A)$ $Gini : \min Gini_A(D)$
--

Main biases:

- IG: favors multivalued attributes.
- GR: may favor unbalanced splits.
- Gini: binary splits; problems with many classes.

Naïve Bayes Classifier

Let $X = (x_1, \dots, x_n)$ be an unlabeled tuple and C_i a possible class.

Bayes' theorem:

$$P(C_i | X) = \frac{P(X | C_i)P(C_i)}{P(X)}$$

Since $P(X)$ is the same for every class:

$$P(C_i | X) \propto P(X | C_i)P(C_i)$$

Prior probability:

$$P(C_i) = \frac{|C_{i,D}|}{|D|}$$

Classification rule:

$$C^* = \arg \max_{C_i} P(C_i | X)$$

equivalently:

$$C^* = \arg \max_{C_i} P(X | C_i)P(C_i)$$

Conditional Independence Assumption

Naïve Bayes assumes attributes are **conditionally independent given the class**:

$$P(X | C_i) = \prod_{k=1}^n P(x_k | C_i)$$

Therefore:

$$Score(C_i | X) = P(C_i) \prod_{k=1}^n P(x_k | C_i)$$

and:

$$C^* = \arg \max_{C_i} \left[P(C_i) \prod_{k=1}^n P(x_k | C_i) \right]$$

Categorical Attributes

For a categorical attribute A_k :

$$P(x_k | C_i) = \frac{|\{x \in C_i : A_k(x) = x_k\}|}{|C_{i,D}|}$$

Continuous Attributes – Gaussian

For a continuous-valued attribute:

$$g(x, \mu, \sigma) = \frac{1}{\sqrt{2\pi}\sigma} \exp \left[-\frac{(x - \mu)^2}{2\sigma^2} \right]$$

Therefore:

$$P(x_k | C_i) = g(x_k, \mu_{k,C_i}, \sigma_{k,C_i})$$

where:

$$\mu_{k,C_i} = \frac{1}{|C_{i,D}|} \sum_{x \in C_i} x_k$$

and σ_{k,C_i} is computed only on tuples belonging to class C_i .

Hence:

$$P(X | C_i) = \prod_{k=1}^n g(x_k, \mu_{k,C_i}, \sigma_{k,C_i})$$

Zero-Probability Problem

If:

$$P(x_k | C_i) = 0$$

then:

$$P(X | C_i) = 0$$

Use the **Laplacian correction**. If the attribute has v possible values:

$$P(x_k = a_j | C_i) = \frac{n_{ij} + 1}{N_i + v}$$

where n_{ij} is the number of class- C_i tuples having value a_j , and $N_i = |C_{i,D}|$.

Naïve Bayes Summary

$P(C_i X) = \frac{P(X C_i)P(C_i)}{P(X)}$
--

$P(X C_i) = \prod_{k=1}^n P(x_k C_i)$

$C^* = \arg \max_{C_i} \left[P(C_i) \prod_{k=1}^n P(x_k C_i) \right]$
--

IF-THEN Rule Classifier

A rule has the form:

$$R : \quad \text{IF } p_1 \wedge p_2 \wedge \dots \wedge p_l \quad \text{THEN } C_i$$

Rule coverage:

$$Coverage(R) = \frac{n_{covers}}{|D|}$$

Rule accuracy:

$$Accuracy(R) = \frac{n_{correct}}{n_{covers}}$$

where n_{covers} is the number of tuples satisfying the antecedent and $n_{correct}$ is the number of covered tuples belonging to the predicted class.

Sequential Covering

Learn rules **one at a time**, considering one target class C_i at a time:

$$RuleSet = \emptyset$$

$$R = \text{LearnOneRule}(D, C_i)$$

$$\text{RuleSet} \leftarrow \text{RuleSet} \cup \{R\}$$

then remove the tuples covered by R and repeat:

$$D \leftarrow D - \{x \in D : R(x) = \text{true}\}$$

until the termination condition is reached.

To learn one rule, start from the most general rule:

$$IF \emptyset \text{ THEN } C_i$$

and greedily add predicates:

$$R' = R \wedge p$$

choosing the predicate p with maximum **FOIL Gain**.

FOIL Gain

Let pos, neg be the positive and negative tuples covered by the current rule R , and pos', neg' those covered after:

$$R' = R \wedge p$$

Then:

$$FOIL_Gain = pos' \left[\log_2 \left(\frac{pos'}{pos' + neg'} \right) - \log_2 \left(\frac{pos}{pos + neg} \right) \right]$$

Select:

$$p^* = \arg \max_p FOIL_Gain(p)$$

The logarithmic term measures improvement in purity; pos' accounts for positive coverage.

FOIL Rule Pruning

$$FOIL_Prune(R) = \frac{pos - neg}{pos + neg}$$

For a candidate pruned rule R' :

$$FOIL_Prune(R') > FOIL_Prune(R) \Rightarrow \text{prune}$$

Pruning removes one conjunct at a time. RIPPER starts from the most recently added conjunct.

Rule Learning Summary

$$\begin{aligned} Coverage(R) &= \frac{n_{covers}}{|D|} \\ Accuracy(R) &= \frac{n_{correct}}{n_{covers}} \\ FOIL_Gain &= pos' \left[\log_2 \frac{pos'}{pos' + neg'} - \log_2 \frac{pos}{pos + neg} \right] \\ FOIL_Prune(R) &= \frac{pos - neg}{pos + neg} \end{aligned}$$

$FOIL_Gain \rightarrow$ build/refine rule, $FOIL_Prune \rightarrow$ simplify

Confusion Matrix Quantities

$$P = TP + FN, \quad N = TN + FP$$

Accuracy:

$$Accuracy = \frac{TP + TN}{P + N}$$

Sensitivity:

$$Sensitivity = \frac{TP}{P} = \frac{TP}{TP + FN}$$

Specificity:

$$Specificity = \frac{TN}{N} = \frac{TN}{TN + FP}$$

Precision and Recall

Precision:

$$Precision = \frac{TP}{TP + FP}$$

Recall:

$$Recall = \frac{TP}{TP + FN} = Sensitivity$$

Precision measures exactness; recall measures completeness.

F_1 and F_β Measures

$$F_1 = \frac{2 Precision Recall}{Precision + Recall}$$

General weighted form:

$$F_\beta = \frac{(1 + \beta^2) Precision Recall}{\beta^2 Precision + Recall}$$

$\beta > 1 \Rightarrow$ more weight to Recall

$\beta < 1 \Rightarrow$ more weight to Precision

Common cases:

F_2 : Recall emphasized, $F_{0.5}$: Precision emphasized

For imbalanced data, prefer F -measure or AUC to plain accuracy.

Bootstrap – .632 Method

Given d tuples, draw d samples **with replacement**.

Probability that a tuple is never selected:

$$P(\text{not selected}) = \left(1 - \frac{1}{d}\right)^d \approx e^{-1} \approx 0.368$$

Hence approximately:

63.2%

of the original tuples occur in the bootstrap training sample and approximately:

36.8%

are left for testing.

After k bootstrap repetitions:

$$Acc(M) = \sum_{i=1}^k [0.632 Acc(M_i)_{test} + 0.368 Acc(M_i)_{train}]$$

Paired Student t -Test

Compare two classifiers using the **same cross-validation partitions**.

For k paired experiments:

$$d_i = \text{err}(M_1)_i - \text{err}(M_2)_i$$

The t statistic is:

$$t = \frac{\overline{\text{err}(M_1)} - \overline{\text{err}(M_2)}}{\sqrt{\text{var}(M_1 - M_2)/k}}$$

Degrees of freedom:

$$DoF = k - 1$$

Null hypothesis:

$$H_0 : \mu_{M_1} = \mu_{M_2}$$

For a two-tailed test at significance α :

$$|t| > t_{\alpha/2, k-1} \Rightarrow \text{reject } H_0$$

The paired t -test assumes approximately Gaussian results and comparable variances.

Different test sets $\Rightarrow$ use a **non-paired t -test**.

Wilcoxon Signed-Rank Test

Non-parametric alternative to the paired t -test.

For each paired observation:

$$d_i = x_i - y_i$$

Rank the absolute differences:

$$|d_i|$$

and restore their signs.

Sum positive and negative ranks:

$$W^+ = \sum_{d_i > 0} \text{rank}(|d_i|)$$

$$W^- = \sum_{d_i < 0} \text{rank}(|d_i|)$$

Check:

$$W^+ + W^- = \frac{n(n+1)}{2}$$

Test statistic:

$$W = \min(W^+, W^-)$$

Null hypothesis:

$$H_0 : \text{median difference} = 0$$

or equivalently equal population medians.

Wilcoxon – Normal Approximation

If:

$$\frac{n(n+1)}{2} > 20$$

use:

$$\mu_W = \frac{n(n+1)}{4}$$

$$\sigma_W = \sqrt{\frac{n(n+1)(2n+1)}{24}}$$

and:

$$z = \frac{W - \mu_W}{\sigma_W}$$

For a two-tailed test, obtain the corresponding two-tailed p -value.

Wilcoxon – Ties

If:

$$d_i = 0$$

ignore the observation and reduce n .

If several absolute differences are equal, assign their **average rank**.

For a tied group of size t , reduce the variance by:

$$\frac{t^3 - t}{48}$$

With several tied groups:

$$\sigma_{W,tie}^2 = \frac{n(n+1)(2n+1)}{24} - \sum_j \frac{t_j^3 - t_j}{48}$$

ROC Curve

ROC coordinates:

$$TPR = \frac{TP}{P} = \frac{TP}{TP + FN}$$

$$FPR = \frac{FP}{N} = \frac{FP}{FP + TN}$$

Therefore each classifier configuration gives:

$$(FPR, TPR)$$

Ideal point:

$$(FPR, TPR) = (0, 1)$$

Diagonal/random behavior:

$$TPR = FPR$$

Generating the ROC Curve

For a probabilistic classifier with score $s(x)$, classify as positive when:

$$s(x) > T$$

Start from a high threshold and progressively decrease T .

For every threshold:

$$T \rightarrow (TP, FP, TN, FN) \rightarrow (FPR, TPR)$$

High threshold:

$$(FPR, TPR) \approx (0, 0)$$

Decrease threshold:

$$TP \uparrow, \quad FP \uparrow$$

Very low threshold:

$$(FPR, TPR) \rightarrow (1, 1)$$

Connect the resulting points to obtain the ROC curve.

AUC – Area Under the ROC Curve

$$AUC = \text{Area under ROC curve}$$

Important reference values:

$$AUC = 1 \Rightarrow \text{perfect classifier}$$

$$AUC = 0.5 \Rightarrow \text{diagonal / worthless classifier}$$

Higher AUC:

$$AUC \uparrow \Rightarrow \text{better classifier}$$

For cross-validation:

$$AUC_1, AUC_2, \dots, AUC_k$$

form an AUC distribution that can be compared between classifiers using a statistical test. and:

Cost of a Classifier

Let:

$$P(p) = \text{prior probability of positive class}$$

$$P(n) = \text{prior probability of negative class}$$

and:

$$C(Y, n) = \text{cost of false positive}$$

$$C(N, p) = \text{cost of false negative}$$

False-negative rate:

$$FNR = \frac{FN}{P} = 1 - TPR$$

False-positive rate:

$$FPR = \frac{FP}{N}$$

A compact form of the misclassification cost is:

$$Cost = P(n) * FPR * C(Y, n) + P(p) * FNR * C(N, p)$$

Equivalently, using raw counts:

$$Cost \propto FPC(Y, n) + FNC(N, p)$$

Iso-Cost Lines

For fixed priors and misclassification costs, iso-cost lines in ROC space have slope:

$$\text{slope} = \frac{P(n) C(Y, n)}{P(p) C(N, p)}$$

Points on the same iso-cost line have the same cost.

Cost decreases moving toward:

$$(0, 1)$$

The minimum-cost operating point is the tangent point between the ROC convex hull and an iso-cost line.

Model Evaluation Summary

$$\begin{aligned} \text{Sensitivity} &= \frac{TP}{TP + FN} \\ \text{Specificity} &= \frac{TN}{TN + FP} \\ \text{Precision} &= \frac{TP}{TP + FP} \\ \text{Recall} &= \frac{TP}{TP + FN} \\ F_1 &= \frac{2PR}{P + R} \\ TPR &= \frac{TP}{P} \\ FPR &= \frac{FP}{N} \end{aligned}$$

where in the F_1 expression:

$$P = \text{Precision}, \quad R = \text{Recall}$$

$$\begin{aligned} t &= \frac{\overline{err(M_1)} - \overline{err(M_2)}}{\sqrt{\text{var}(M_1 - M_2)/k}} \\ W &= \min(W^+, W^-) \\ \mu_W &= \frac{n(n+1)}{4} \\ \sigma_W^2 &= \frac{n(n+1)(2n+1)}{24} \\ AUC &\in [0, 1] \end{aligned}$$

“tex **AdaBoost**

Given d labeled tuples, initialize:

$$w_j = \frac{1}{d}, \quad j = 1, \dots, d$$

Weighted error of classifier M_i :

$$\text{error}(M_i) = \sum_{j=1}^d w_j \text{err}(X_j)$$

where:

$$\text{err}(X_j) = \begin{cases} 1, & M_i(X_j) \neq y_j \\ 0, & M_i(X_j) = y_j \end{cases}$$

For a **correctly classified** tuple:

$$w_j \leftarrow w_j \frac{\text{error}(M_i)}{1 - \text{error}(M_i)}$$

Thus correctly classified tuples decrease their weight; after normalization, misclassified tuples obtain larger relative weight.

Normalize:

$$w_j \leftarrow \frac{w_j}{\sum_{\ell=1}^d w_\ell}$$

so that:

$$\sum_{j=1}^d w_j = 1$$

Classifier vote weight:

$$\alpha_i = \log \left(\frac{1 - \text{error}(M_i)}{\text{error}(M_i)} \right)$$

Hence:

$$\text{error}(M_i) \downarrow \Rightarrow \alpha_i \uparrow$$

Final weighted vote:

$$M^*(X) = \arg \max_c \sum_{i: M_i(X)=c} \alpha_i$$

AdaBoost Summary

$$\begin{aligned} w_j^{(0)} &= \frac{1}{d} \\ \text{error}(M_i) &= \sum_j w_j \text{err}(X_j) \\ w_j^{\text{correct}} &\leftarrow w_j \frac{\text{error}(M_i)}{1 - \text{error}(M_i)} \\ \alpha_i &= \log \frac{1 - \text{error}(M_i)}{\text{error}(M_i)} \end{aligned}$$

misclassified tuples $\Rightarrow$ higher relative weight

better classifier $\Rightarrow$ higher voting weight

SMOTE

For a minority tuple x_i , choose one of its k nearest minority-class neighbors x_{z_i} .

$$\Delta x = x_{z_i} - x_i$$

Choose:

$$\delta \sim U(0, 1)$$

Generate:

$$x_{new} = x_i + \delta(x_{z_i} - x_i)$$

with:

$$0 \leq \delta \leq 1$$

Thus x_{new} lies on the line segment between x_i and the selected minority neighbor x_{z_i} .

$$SMOTE : \quad x_{new} = x_i + \delta(x_{z_i} - x_i)$$

Synthetic minority samples are generated instead of simply duplicating existing observations.

5 – Clustering

Hopkins Statistic

Tests whether the dataset has a natural **clustering tendency** or is approximately uniform.

Sample $n \ll N$ real points $p_i \in D$:

$$x_i = \min_{v \in D} \{dist(p_i, v)\}$$

Generate n synthetic points q_i from a uniform distribution over the same data space:

$$y_i = \min_{v \in D} \{dist(q_i, v)\}$$

Hopkins statistic:

$$H = \frac{\sum_{i=1}^n y_i}{\sum_{i=1}^n x_i + \sum_{i=1}^n y_i}$$

Interpretation:

$H \simeq 0.5 \Rightarrow$ uniform / no clustering tendency

$0.7 \leq H \leq 0.9 \Rightarrow$ significant clustering tendency

$H \simeq 1 \Rightarrow$ strongly separated clusters

$H > 0.75 \Rightarrow$ clustering tendency (90% confidence)

$H_0 : D$ uniform, $H_1 : D$ non-uniform

Partitioning Methods

Objective function:

$$E = \sum_{i=1}^k \sum_{p \in C_i} \|p - c_i\|^2$$

where c_i is the representative of C_i .

$$\min E$$

Global optimization requires enumeration of all partitions $\Rightarrow$ heuristic methods are used.

k-Means

Centroid:

$$c_i = \frac{1}{|C_i|} \sum_{p \in C_i} p$$

Assignment:

$$p \rightarrow \arg \min_i \|p - c_i\|$$

Objective:

$$\min \sum_{i=1}^k \sum_{p \in C_i} \|p - c_i\|^2$$

Computational cost:

$$O(tkn)$$

$n = \# \text{objects}, \quad k = \# \text{clusters}, \quad t = \# \text{iterations}$

Usually $k, t \ll n$.

PAM / k-Medoids

Representative = **medoid**, i.e. an actual object of the dataset.

For medoid O and non-medoid R :

$$O \leftrightarrow R$$

Swap cost:

$$S = E_{\text{new}} - E_{\text{old}}$$

Accept if:

$$S < 0$$

Computational cost per iteration:

$$O(k(n-k)^2)$$

Hierarchical Linkage Measures

General inter-cluster distance:

$$d(C_1, C_2) = \text{op}\{d(x, y) : x \in C_1, y \in C_2\}$$

Single Link (nearest neighbour):

$$d_{\min}(C_i, C_j) = \min_{\substack{p \in C_i \\ p' \in C_j}} \|p - p'\|$$

Complete Link (furthest neighbour):

$$d_{\max}(C_i, C_j) = \max_{\substack{p \in C_i \\ p' \in C_j}} \|p - p'\|$$

Mean / Centroid Link:

$$d_{\text{mean}}(C_i, C_j) = \|m_i - m_j\|$$

where m_i, m_j are the cluster centroids.

Average Link:

$$d_{\text{avg}}(C_i, C_j) = \frac{1}{n_i n_j} \sum_{p \in C_i} \sum_{p' \in C_j} \|p - p'\|$$

Remember:

single = min, complete = max, average = avg

BIRCH – Clustering Feature

A subcluster is summarized by:

$$CF = (N, LS, SS)$$

with

$$N = |C|$$

$$LS = \sum_{i=1}^N x_i$$

$$SS = \sum_{i=1}^N x_i^2$$

Centroid:

$$x_0 = \frac{LS}{N}$$

Radius:

$$R = \sqrt{\frac{N SS - LS^2}{N^2}}$$

equivalently

$$R = \sqrt{\frac{\sum_{i=1}^N (x_i - x_0)^2}{N}}$$

Diameter:

$$D = \sqrt{\frac{2N SS - 2LS^2}{N(N-1)}}$$

equivalently

$$D = \sqrt{\frac{\sum_{i=1}^N \sum_{j=1}^N (x_i - x_j)^2}{N(N-1)}}$$

CF additivity:

$$CF_1 + CF_2 = (N_1 + N_2, LS_1 + LS_2, SS_1 + SS_2)$$

New point x :

$$N' = N + 1, \quad LS' = LS + x, \quad SS' = SS + x^2$$

CF-tree parameters:

B = branching factor

L = max leaf entries

T = max leaf-subcluster diameter

Insertion condition:

$$D(C \cup \{x\}) \leq T \Rightarrow x \text{ absorbed into } C$$

otherwise a new subcluster is created; if the node is full, it is split.

BIRCH computational cost:

$$O(n)$$

DENCLUE

Density is modeled through kernel density estimation. Given points $x_1, \dots, x_n$:

$$\hat{f}_h(x) = \frac{1}{nh} \sum_{i=1}^n K\left(\frac{x - x_i}{h}\right)$$

where h is the kernel bandwidth.

Kernel properties:

$$\int_{-\infty}^{+\infty} K(u) du = 1$$

$$K(-u) = K(u)$$

Gaussian kernel:

$$K_{\text{Gauss}}\left(\frac{x - x_i}{h}\right) = \frac{1}{\sqrt{2\pi}} e^{-\frac{(x - x_i)^2}{2h^2}}$$

Square kernel:

$$K_{\text{Square}}\left(\frac{x - x_i}{h}\right) = \begin{cases} 0, & \left|\frac{x - x_i}{h}\right| > 1 \\ 1, & \text{otherwise} \end{cases}$$

Density attractor x^* must satisfy:

$$\hat{f}(x^*) \geq \xi$$

where ξ is the density/noise threshold.

Hill-Climbing

Initialization:

$$x^0 = x$$

Update rule:

$$x^{j+1} = x^j + \delta \frac{\nabla \hat{f}(x^j)}{\|\nabla \hat{f}(x^j)\|}$$

where δ controls the convergence step.

For Gaussian kernel:

$$\nabla \hat{f}(x) = \frac{1}{h^{d+2}n} \sum_{i=1}^n K\left(\frac{x - x_i}{h}\right) (x_i - x)$$

Hill-climbing stops when:

$$\hat{f}(x^{k+1}) < \hat{f}(x^k)$$

and

$$x^* = x^k$$

is the density attractor.

Outlier condition:

$$\hat{f}(x^*) < \xi \Rightarrow x \text{ is noise/outlier}$$

Two attractors belong to the same arbitrary-shape cluster if they are connected by a path P such that:

$$\forall p \in P : \hat{f}(p) \geq \xi$$

More formally:

$$\forall x \in C \exists x^* \in X : \hat{f}(x^*) \geq \xi$$

$$\forall x_1^*, x_2^* \in X \exists P : x_1^* \rightsquigarrow x_2^* \quad \wedge \quad \forall p \in P, \hat{f}(p) \geq \xi$$

DENCLUE Grid Optimization

Minimal bounding hyperrectangle is divided into d -dimensional hypercubes with side:

$$2h$$

For each populated cube c store:

$$N_c$$

$$LS_c = \sum_{x \in c} x$$

Cube mean:

$$\text{mean}(c) = \frac{1}{N_c} \sum_{x \in c} x$$

Using Gaussian kernel:

$$h = \sigma$$

Two populated cubes c_1, c_2 are connected if:

$$d(\text{mean}(c_1), \text{mean}(c_2)) < 4\sigma$$

Highly populated cubes:

$$C_{sp} = \{c \in C_p \mid N_c \geq \xi_c\}$$

Relevant cubes:

$$C_r = C_{sp} \cup \{c \in C_p \mid \exists c_s \in C_{sp} : \text{connection}(c_s, c)\}$$

DENCLUE computational cost with optimization:

$$O(N \log N)$$

Parameters:

$$h = \text{kernel bandwidth}$$

$$\xi = \text{minimum significant density}$$

“tex **Clustering Evaluation Metrics**

Two families:

$$\text{Extrinsic} \Rightarrow \text{requires ground truth}$$

$$\text{Intrinsic} \Rightarrow \text{no ground truth}$$

BCubed Precision and Recall

Let

$$L(o_i) = \text{ground-truth class of } o_i$$

$$C(o_i) = \text{cluster assigned to } o_i$$

Pairwise correctness:

$$\text{Correctness}(o_i, o_j) = \begin{cases} 1, & (L(o_i) = L(o_j)) \iff (C(o_i) = C(o_j)) \\ 0, & \text{otherwise} \end{cases}$$

BCubed precision:

$$\text{Precision}_{BCubed} = \frac{1}{n} \sum_{i=1}^n \frac{\sum_{\substack{o_j: j \neq i \\ C(o_i) = C(o_j)}} \text{Correctness}(o_i, o_j)}{|\{o_j \mid j \neq i, C(o_i) = C(o_j)\}|}$$

Precision considers objects in the **same cluster**.

BCubed recall:

$$\text{Recall}_{BCubed} = \frac{1}{n} \sum_{i=1}^n \frac{\sum_{\substack{o_j: j \neq i \\ L(o_i) = L(o_j)}} \text{Correctness}(o_i, o_j)}{|\{o_j \mid j \neq i, L(o_i) = L(o_j)\}|}$$

Recall considers objects in the **same ground-truth class**.

Ideal case:

$$\text{Precision} = \text{Recall} = 1$$

Silhouette Coefficient

Intrinsic measure of **compactness** + **separation**.

For object $o \in C_i$:

Intra-cluster distance:

$$a(o) = \frac{\sum_{\substack{o' \in C_i \\ o' \neq o}} \text{dist}(o, o')}{|C_i| - 1}$$

Desired:

$$a(o) \text{ small}$$

Nearest-cluster distance:

$$b(o) = \min_{\substack{C_j \\ j \neq i}} \left\{ \frac{\sum_{o' \in C_j} \text{dist}(o, o')}{|C_j|} \right\}$$

Desired:

$$b(o) \text{ large}$$

Silhouette:

$$s(o) = \frac{b(o) - a(o)}{\max\{a(o), b(o)\}}$$

Range:

$$-1 \leq s(o) \leq 1$$

Interpretation:

$$s(o) \simeq 1 \Rightarrow \text{well clustered}$$

$$s(o) \simeq 0 \Rightarrow \text{boundary / ambiguous point}$$

$$s(o) < 0 \Rightarrow \text{closer to another cluster}$$

Cluster-level silhouette:

$$S(C_i) = \frac{1}{|C_i|} \sum_{o \in C_i} s(o)$$

Overall clustering quality:

$$S = \frac{1}{n} \sum_{o \in D} s(o)$$

Important limitation:

$$\text{Silhouette is reliable mainly for convex clusters}$$

Thus it is suitable for k -means-like clusters, but must be used carefully with arbitrary-shaped clusters such as DBSCAN outputs.

Bi-Clustering

A bi-cluster is a submatrix $I \times J$.

Constant values:

$$e_{ij} = c$$

Constant on rows:

$$e_{ij} = c + \alpha_i$$

Symmetrically, constant on columns:

$$e_{ij} = c + \beta_j$$

Coherent values:

$$e_{ij} = c + \alpha_i + \beta_j$$

Equivalent condition:

$$e_{i_1 j_1} - e_{i_2 j_1} = e_{i_1 j_2} - e_{i_2 j_2}$$

Coherent evolution:

$$(e_{i_1 j_1} - e_{i_1 j_2})(e_{i_2 j_1} - e_{i_2 j_2}) \geq 0$$

Only the direction of change matters.

δ -Clustering

For candidate submatrix $I \times J$:

Row mean:

$$e_{iJ} = \frac{1}{|J|} \sum_{j \in J} e_{ij}$$

Column mean:

$$e_{IJ} = \frac{1}{|I|} \sum_{i \in I} e_{ij}$$

Submatrix mean:

$$e_{IJ} = \frac{1}{|I||J|} \sum_{i \in I} \sum_{j \in J} e_{ij}$$

Residue:

$$r_{ij} = e_{ij} - e_{iJ} - e_{IJ} + e_{IJ}$$

Mean Squared Residue:

$$H(I \times J) = \frac{1}{|I||J|} \sum_{i \in I} \sum_{j \in J} r_{ij}^2$$

δ -bi-cluster condition:

$$H(I \times J) \leq \delta$$

Perfect coherent bi-cluster:

$$\delta = 0 \Rightarrow H(I \times J) = 0$$

Row contribution:

$$d(i) = \frac{1}{|J|} \sum_{j \in J} r_{ij}^2$$

Column contribution:

$$d(j) = \frac{1}{|I|} \sum_{i \in I} r_{ij}^2$$

Deletion phase:

$$H > \delta \Rightarrow \text{remove row/column with largest } d$$

Addition phase:

$$\text{add excluded row/column with smallest } d$$

only if after insertion:

$$H(I \times J) \leq \delta$$

δ -pCluster

For every 2×2 submatrix:

$$\begin{pmatrix} e_{i_1 j_1} & e_{i_1 j_2} \\ e_{i_2 j_1} & e_{i_2 j_2} \end{pmatrix}$$

p-score:

$$p = |(e_{i_1 j_1} - e_{i_2 j_1}) - (e_{i_1 j_2} - e_{i_2 j_2})|$$

δ -pCluster condition:

$$p \leq \delta \quad \forall 2 \times 2 \text{ submatrices}$$

Perfect pattern:

$$\delta = 0 \Rightarrow e_{i_1 j_1} - e_{i_2 j_1} = e_{i_1 j_2} - e_{i_2 j_2}$$

Monotonicity:

$$I \times J \text{ is a } \delta\text{-pCluster} \Rightarrow I' \times J' \text{ is too}$$

for every

$$I' \subseteq I, \quad J' \subseteq J, \quad |I'|, |J'| \geq 2.$$

Hence:

$$\text{invalid submatrix} \Rightarrow \text{all containing supermatrices invalid}$$

Maximal δ -pCluster:

$$\nexists I' \times J' \supset I \times J \text{ satisfying } p \leq \delta$$

MaPle: pattern-growth enumeration of all maximal δ -pClusters. **SimRank**

In-neighbourhood:

$$I(v) = \{u \mid (u, v) \in E\}$$

Structural similarity:

$$s(u, v) = \frac{C}{|I(u)||I(v)|} \sum_{x \in I(u)} \sum_{y \in I(v)} s(x, y)$$

with

$$s(u, u) = 1, \quad 0 < C < 1$$

Initialization:

$$s_0(u, v) = \begin{cases} 1, & u = v \\ 0, & u \neq v \end{cases}$$

Iterative computation:

$$s_{k+1}(u, v) = \frac{C}{|I(u)||I(v)|} \sum_{x \in I(u)} \sum_{y \in I(v)} s_k(x, y)$$

Similarity to dissimilarity:

$$d(u, v) = 1 - s(u, v)$$

Computational cost:

$$O(Kn^2d^2)$$

where K is the number of iterations and d^2 represents the average product of in-neighbourhood sizes.

Random Walk and Meeting Distance

For a tour

$$t = \langle w_1, \dots, w_k \rangle$$

tour length:

$$\ell(t) = k - 1$$

Tour probability:

$$P[t] = \prod_{i=1}^{k-1} \frac{1}{|O(w_i)|}$$

Expected distance:

$$d(u, v) = \sum_{t: u \rightsquigarrow v} P[t] \ell(t)$$

where v is reached for the first time at the end of the tour. In the derived graph G^2 , a state is a pair (u, v) . Singleton states:

$$(x, x)$$

represent the meeting of the two surfers.

Expected Meeting Distance:

$$m(u, v) = \sum_{t: (u, v) \rightsquigarrow (x, x)} P[t] \ell(t)$$

Problem:

$$\text{no meeting} \Rightarrow \ell(t) = \infty$$

Use a bounded transformation:

$$f(z) = C^z, \quad 0 < C < 1$$

Expected- f meeting similarity:

$$s'(u, v) = \sum_{t: (u, v) \rightsquigarrow (x, x)} P[t] C^{\ell(t)}$$

Thus:

$$\ell = 0 \Rightarrow C^\ell = 1, \quad \ell \rightarrow \infty \Rightarrow C^\ell \rightarrow 0$$

and

$$s(u, v) = s'(u, v)$$

when the random walks travel through back-edges.

Graph Cuts and Sparsity

For a cut

$$V = S \cup T, \quad S \cap T = \emptyset$$

cut set:

$$E(S, T) = \{(u, v) \in E \mid u \in S, v \in T\}$$

Cut sparsity:

$$\Phi(S, T) = \frac{|E(S, T)|}{\min\{|S|, |T|\}}$$

Sparsest cut:

$$\min_{S, T} \Phi(S, T)$$

The denominator penalizes highly unbalanced cuts.

Modularity

For a clustering into k clusters:

$$Q = \sum_{i=1}^k \left[\frac{l_i}{|E|} - \left(\frac{d_i}{2|E|} \right)^2 \right]$$

where

$$l_i = \#\{\text{edges internal to } C_i\}$$

$$d_i = \sum_{v \in C_i} \deg(v)$$

and

$$\sum_{v \in V} \deg(v) = 2|E|$$

Interpretation:

$$\frac{l_i}{|E|} = \text{observed internal-edge fraction}$$

$$\left(\frac{d_i}{2|E|} \right)^2 = \text{expected fraction under random links}$$

Optimal clustering:

$$\max Q$$

SCAN

Immediate neighbourhood:

$$\Gamma(v) = \{w \in V \mid (v, w) \in E\} \cup \{v\}$$

Structural similarity:

$$\sigma(v, w) = \frac{|\Gamma(v) \cap \Gamma(w)|}{\sqrt{|\Gamma(v)| |\Gamma(w)|}}$$

ε -neighbourhood:

$$N_\varepsilon(v) = \{w \in \Gamma(v) \mid \sigma(v, w) \geq \varepsilon\}$$

Core vertex:

$$CORE_{\varepsilon, \mu}(v) \iff |N_\varepsilon(v)| \geq \mu$$

Direct structural reachability:

$$DirREACH_{\varepsilon, \mu}(v, w) \iff CORE_{\varepsilon, \mu}(v) \wedge w \in N_\varepsilon(v)$$

Structural connectivity:

$$\boxed{CONNECT_{\varepsilon,\mu}(v,w) \iff \exists u \in V : REACH_{\varepsilon,\mu}(u,v) \wedge REACH_{\varepsilon,\mu}(u,w)}$$

hub $\Rightarrow$ non-member connecting many clusters

Cluster conditions:

outlier $\Rightarrow$ non-member connecting few clusters

Computational cost:

$$\boxed{\forall v, w \in C : CONNECT_{\varepsilon,\mu}(v,w)}$$

$$\boxed{O(|E|)}$$

$$\boxed{v \in C \wedge REACH_{\varepsilon,\mu}(v,w) \Rightarrow w \in C}$$

For sparse graphs:

SCAN output:

$$\boxed{O(|E|) \simeq O(|V|)}$$

$$\boxed{\text{clusters} + \text{hubs} + \text{outliers}}$$

6 – Association Rules

Support and Confidence

Absolute support:

$$\sigma(X) = |\{T \in D : X \subseteq T\}|$$

Relative support:

$$supp(X) = \frac{\sigma(X)}{|D|}$$

For a rule $X \Rightarrow Y$:

$$supp(X \Rightarrow Y) = P(X \cup Y)$$

$$conf(X \Rightarrow Y) = P(Y | X) = \frac{supp(X \cup Y)}{supp(X)}$$

Frequent itemset condition:

$$supp(X) \geq minsup$$

Strong rule:

$$supp(X \cup Y) \geq minsup, \quad conf(X \Rightarrow Y) \geq minconf$$

Binomial Coefficient

Number of k -itemsets from n items:

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

Total number of non-empty itemsets:

$$\sum_{k=1}^n \binom{n}{k} = 2^n - 1$$

Closed and Maximal Itemsets

Support anti-monotonicity:

$$X \subseteq Y \Rightarrow supp(Y) \leq supp(X)$$

X is **closed** iff:

$$X \text{ frequent}$$

and

$$\nexists Y \supset X : supp(Y) = supp(X)$$

X is **maximal** iff:

$$X \text{ frequent}$$

and

$$\nexists Y \supset X : Y \text{ frequent}$$

$$\text{Maximal} \Rightarrow \text{Closed}$$

Closed $\Rightarrow$ lossless compression.

Maximal $\Rightarrow$ support information may be lost.

Apriori

Apriori property:

$$X \text{ frequent} \Rightarrow \forall S \subset X, S \text{ frequent}$$

Equivalent pruning rule:

$$S \text{ infrequent} \Rightarrow \forall X \supset S, X \text{ infrequent}$$

Notation:

$$L_k = \{\text{frequent } k\text{-itemsets}\}$$

$$C_k = \{\text{candidate } k\text{-itemsets}\}$$

Join condition for $l_1, l_2 \in L_{k-1}$:

$$l_1[1] = l_2[1], \dots, l_1[k-2] = l_2[k-2]$$

$$l_1[k-1] < l_2[k-1]$$

Candidate:

$$c = \{l_1[1], \dots, l_1[k-1], l_2[k-1]\}$$

Pruning:

$$\exists S \subset c, |S| = k-1, S \notin L_{k-1} \Rightarrow c \text{ is pruned}$$

Rule generation from frequent itemset I :

$$s \subset I, \quad s \neq \emptyset$$

$$s \Rightarrow (I - s)$$

$$conf = \frac{supp(I)}{supp(s)}$$

FP-Growth

Global frequency ordering:

$$supp(i_1) \geq supp(i_2) \geq \dots \geq supp(i_m)$$

Each transaction is reordered according to this global order.

FP-Growth pipeline:

$$D \rightarrow FP\text{-tree} \rightarrow \text{conditional pattern base}$$

$$\rightarrow \text{conditional FP-tree} \rightarrow \text{frequent patterns}$$

Support of suffix item x :

$$supp(x) = \sum_{\text{nodes of } x} count(x)$$

Conditional support:

$$supp(a | x) = \sum_{\substack{\text{prefix paths} \\ \text{containing } a}} count(\text{path})$$

Keep a in the conditional FP-tree iff:

$$supp(a | x) \geq minsup$$

Original FP-tree / prefix paths:

$$count_{\text{prefix}} = count_{\text{original}}$$

Conditional FP-tree:

$$count_{\text{conditional}} = \text{recomputed conditional frequency}$$

Lift

$$lift(A \Rightarrow B) = \frac{conf(A \Rightarrow B)}{supp(B)}$$

Equivalently:

$$lift(A, B) = \frac{P(A \cap B)}{P(A)P(B)}$$

$lift = 1 \Rightarrow$ independence

$lift > 1 \Rightarrow$ positive association

$lift < 1 \Rightarrow$ negative association

$$lift(A \Rightarrow B) = lift(B \Rightarrow A)$$

Null-Invariant Measures

All-confidence:

$$all_conf(A, B) = \frac{supp(A \cup B)}{\max\{supp(A), supp(B)\}}$$

$$= \min\{P(A | B), P(B | A)\}$$

Max-confidence:

$$max_conf(A, B) = \max\{P(A | B), P(B | A)\}$$

Kulczynski:

$$Kulc(A, B) = \frac{1}{2} [P(A | B) + P(B | A)]$$

Cosine:

$$cosine(A, B) = \frac{supp(A \cap B)}{\sqrt{supp(A)supp(B)}}$$

Equivalently:

$$cosine(A, B) = \sqrt{P(A | B)P(B | A)}$$

all_conf , max_conf , $Kulc$, $cosine$ are null-invariant

$lift$, χ^2 are not null-invariant

Imbalance Ratio

$$IR(A, B) = \frac{|supp(A) - supp(B)|}{supp(A) + supp(B) - supp(A \cup B)}$$

$IR \simeq 0 \Rightarrow$ balanced

$IR \simeq 1 \Rightarrow$ strongly imbalanced

Use together:

$$Kulczynski + IR$$

to distinguish association strength from imbalance between the two itemsets. **Colossal Patterns**

Downward-closure explosion for a frequent pattern of size n :

$$N_{subpatterns} = 2^n - 1$$

Number of patterns of size k :

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

For $k = n/2$:

$$\binom{n}{n/2} \approx \frac{2^n}{\sqrt{n}}$$

Example:

$$\binom{40}{20} \approx 1.37 \times 10^{11}$$

τ -core pattern

Let $\beta \subseteq \alpha$. β is a τ -core pattern of α iff:

$$\frac{|D_\alpha|}{|D_\beta|} \geq \tau, \quad 0 < \tau \leq 1$$

where:

$$D_\alpha = \{\text{transactions supporting } \alpha\}$$

Since $\beta \subseteq \alpha$:

$$D_\alpha \subseteq D_\beta$$

(d, τ) -robustness

d is the maximum number of items removable from α while the resulting pattern remains a τ -core pattern.

Number of core patterns:

$$N_{core} = \Omega(2^d)$$

Pattern distance

$$Dist(\alpha, \beta) = 1 - \frac{|D_\alpha \cap D_\beta|}{|D_\alpha \cup D_\beta|}$$

Core patterns of the same colossal pattern lie inside a bounding ball:

$$Dist(\alpha, \beta) \leq 1 - \frac{1}{\frac{2}{\tau} - 1}$$

Define:

$$r(\tau) = 1 - \frac{1}{\frac{2}{\tau} - 1}$$

Redundancy

For two patterns p and q :

$$R(p, q) = S(p) + S(q) - S(p, q)$$

where:

$$S(p) = \text{significance of pattern } p$$

High redundancy:

$$R(p, q) \uparrow \Rightarrow \text{large overlap / similar information}$$

Low redundancy:

$$R(p, q) \downarrow \Rightarrow \text{more complementary information}$$

Redundancy-aware top- k mining:

$$\text{high significance} + \text{low redundancy}$$

AprioriSome – Hit Ratio

At level k :

$$hit_k = \frac{|L_k|}{|C_k|}$$

where:

$L_k = \{\text{frequent } k\text{-sequences}\}, \quad C_k = \{\text{candidate } k\text{-sequences}\}$

Interpretation:

$hit_k \uparrow \Rightarrow |L_k| \simeq |C_k| \Rightarrow \text{most candidates are frequent}$

Hence larger hit_k allows a larger jump in the forward phase.

$$hit_k < 0.666 \Rightarrow next(k) = k + 1$$

$$0.666 \leq hit_k < 0.75 \Rightarrow next(k) = k + 2$$

$$0.75 \leq hit_k < 0.80 \Rightarrow next(k) = k + 3$$

$$0.80 \leq hit_k < 0.85 \Rightarrow next(k) = k + 4$$

$$hit_k \geq 0.85 \Rightarrow next(k) = k + 5$$

FreeSpan – Projection Enlargement

For **parallel projection**, let l be the average number of frequent items in a transaction.

A transaction is projected into $l - 1$ projected databases.

Total projected size generated by one transaction:

$$1 + 2 + \dots + (l - 1) = \frac{l(l - 1)}{2}$$

Relative enlargement with respect to the original transaction size l :

$$\frac{\frac{l(l-1)}{2}}{l} = \frac{l-1}{2}$$

Hence:

$$|D_{\text{proj}}| \approx \frac{l-1}{2} |D|$$

Parallel projection:

few scans $\Rightarrow$ high memory / replicated sequences

Partition projection:

one projected DB per sequence at a time $\Rightarrow$ lower memory/more scans

7 – Outlier Analysis

Maximum Likelihood – Gaussian Model

Assume:

$$X \sim \mathcal{N}(\mu, \sigma^2)$$

Probability density of a point:

$$P(x_i) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(x_i - \mu)^2}{2\sigma^2}\right)$$

Likelihood of the whole sample:

$$L(\mu, \sigma \mid X) = \prod_{i=1}^n P(x_i)$$

MLE estimates:

$$\hat{\mu} = \bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

$$\hat{\sigma}^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2$$

3 σ rule:

$$|x - \mu| > 3\sigma \Rightarrow \text{outlier}$$

Grubbs' Test

Detects an extreme minimum or maximum.

Test statistic:

$$G = \frac{\max_i |Y_i - \bar{Y}|}{s}$$

Critical value:

$$G_{\text{crit}} = \frac{N-1}{\sqrt{N}} \sqrt{\frac{t_{\alpha/(2N), N-2}^2}{N-2 + t_{\alpha/(2N), N-2}^2}}$$

Decision:

$$G \geq G_{\text{crit}} \Rightarrow \text{outlier}$$

$$G < G_{\text{crit}} \Rightarrow \text{not outlier}$$

Mahalanobis Distance

For multivariate objects, let $\bar{o}$ be the mean vector and S the covariance matrix.

$$MDist^2(o, \bar{o}) = (o - \bar{o})^T S^{-1} (o - \bar{o})$$

Conceptually:

center $\rightarrow$ decorrelate $\rightarrow$ scale $\rightarrow$ Euclidean distance

Equivalent transformed space:

$$z = S^{-1/2} (o - \bar{o})$$

$$MDist(o, \bar{o}) = \|z\|_2$$

Use the scalar distances $\{MDist(o, \bar{o})\}$ for univariate outlier detection.

Kernel Density Estimation

Non-parametric density estimate:

$$\hat{f}_h(x) = \frac{1}{nh} \sum_{i=1}^n K\left(\frac{x - x_i}{h}\right)$$

where:

$$K(\cdot) = \text{kernel}, \quad h > 0 = \text{bandwidth}$$

Interpretation:

$$\hat{f}_h(x) \text{ high} \Rightarrow \text{normal}$$

$$\hat{f}_h(x) \text{ low} \Rightarrow \text{possible outlier}$$

Distance-Based Outlier Detection

An object o is a $DB(r, \pi)$ outlier if too few objects lie within radius r :

$$\frac{|\{o' \in D : dist(o, o') \leq r\}|}{|D|} \leq \pi$$

Equivalent k -NN formulation:

$$k = \lceil \pi |D| \rceil$$

$$dist(o, o_k) > r \Rightarrow o \text{ is an outlier}$$

Parameters:

$$r = \text{distance threshold}, \quad \pi = \text{fraction threshold}$$

Grid CELL – Pruning Conditions

Partition the space into hypercubes with diagonal:

$$\frac{r}{2}$$

Let:

$$a = |C|$$

$$b_1 = \# \text{ objects in level-1 cells}$$

$$b_2 = \# \text{ objects in level-2 cells}$$

Level-1 pruning:

$$a + b_1 > \lceil \pi n \rceil$$

$$\Rightarrow \text{all objects in } C \text{ are NOT outliers}$$

Level-2 pruning:

$$a + b_1 + b_2 < \lceil \pi n \rceil + 1$$

$$\Rightarrow \text{all objects in } C \text{ ARE outliers}$$

Both implications are one-way only.

Local Distance-Based Outlierness

k -nearest neighbors:

$$N_k(x_i)$$

Average distance to the k nearest neighbors:

$$D_k(x_i) = \frac{1}{k} \sum_{j \in N_k(x_i)} \|x_i - x_j\|$$

Outlierness:

$$O_k(x_i) = \frac{D_k(x_i)}{\frac{1}{k} \sum_{j \in N_k(x_i)} D_k(x_j)}$$

Interpretation:

$$O_k(x_i) \approx 1 \Rightarrow \text{normal}$$

$$O_k(x_i) > 1 \Rightarrow \text{increasing outlieriness}$$

Parameters:

$$k, \quad \text{outlieriness threshold}$$

Local Reachability / LOF

k -distance:

$$\text{dist}_k(o) = \text{distance to the } k\text{-th NN}$$

k -distance neighborhood:

$$N_k(o) = \{o' \in D : \text{dist}(o, o') \leq \text{dist}_k(o)\}$$

Reachability distance:

$$\text{reachdist}_k(o' \leftarrow o) = \max\{\text{dist}_k(o'), \text{dist}(o, o')\}$$

Local reachability density:

$$\text{lrd}_k(o) = \frac{|N_k(o)|}{\sum_{o' \in N_k(o)} \text{reachdist}_k(o' \leftarrow o)}$$

Local Outlier Factor:

$$\text{LOF}_k(o) = \frac{1}{|N_k(o)|} \sum_{o' \in N_k(o)} \frac{\text{lrd}_k(o')}{\text{lrd}_k(o)}$$

Interpretation:

$$\text{LOF}_k(o) \approx 1 \Rightarrow \text{normal}$$

$$\text{LOF}_k(o) \gg 1 \Rightarrow \text{outlier}$$

CBLOF

Sort clusters by decreasing cardinality:

$$|C_1| \geq |C_2| \geq \dots \geq |C_k|$$

Boundary b between large and small clusters is determined using:

$$\sum_{i=1}^b |C_i| \geq |D|\alpha$$

or

$$\frac{|C_b|}{|C_{b+1}|} \geq \beta$$

Large and small clusters:

$$LC = \{C_i : i \leq b\}$$

$$SC = \{C_i : i > b\}$$

CBLOF:

$$\text{CBLOF}(o) = \begin{cases} |C_i| \min_{C_j \in LC} \text{distance}(o, C_j), & C_i \in SC \\ |C_i| \text{distance}(o, C_i), & C_i \in LC \end{cases}$$

High CBLOF:

$$\Rightarrow \text{high outlieriness}$$

Parameters:

$$\alpha, \beta, \quad \text{clustering parameters, CBLOF threshold}$$

Nearest Neighbor Description (NN-d)

For a test object z :

$$NN^{tr}(z) = \text{nearest training object to } z$$

Acceptance function:

$$f_{NN^{tr}}(z) = \mathbb{I} \left(\frac{\|z - NN^{tr}(z)\|}{\|NN^{tr}(z) - NN^{tr}(NN^{tr}(z))\|} \leq 1 \right)$$

Interpretation:

$$\frac{d(z, NN(z))}{d(NN(z), NN(NN(z)))} \leq 1 \Rightarrow \text{normal}$$

$$> 1 \Rightarrow \text{outlier}$$

Score Contextual Outlier

Let U be the mixture model on contextual attributes and V the mixture model on behavioral attributes.

Conditional mapping:

$$p(V_i | U_j)$$

where $p(V_i | U_j)$ is the probability that an object belonging to contextual cluster U_j exhibits behavior V_i .

Contextual compatibility score:

$$S(o) = \sum_{U_j} p(o \in U_j) \sum_{V_i} p(o \in V_i) p(V_i | U_j)$$

Small compatibility between likely context and observed behavior $\Rightarrow$ possible contextual outlier.

High-Dimensional Outlier Detection

HilOut – full-dimensional approach

For object o , let

$$nn_1(o), \dots, nn_k(o)$$

be its k nearest neighbors.

Outlier weight:

$$w_k(o) = \sum_{i=1}^k \text{dist}(o, nn_i(o))$$

Rank objects by decreasing $w_k(o)$:

$$w_k(o_{(1)}) \geq w_k(o_{(2)}) \geq \dots$$

Top- l objects:

$$\text{largest } w_k(o) \Rightarrow \text{outlier}$$

Dimensionality Reduction – PCA

For outlier detection, low-variance principal components may be preferred:

$$\text{Var}(PC_j) \text{ small}$$

Normal objects are concentrated along such components; large deviations may reveal outliers.

Subspace Outlier Detection

Divide each selected dimension into φ **equi-depth** regions.

Fraction associated with each 1-D region:

$$f = \frac{1}{\varphi}$$

For a k -dimensional cell and n total objects, assuming independent dimensions:

$$E[n(C)] = f^k n = \left(\frac{1}{\varphi}\right)^k n$$

Standard deviation:

$$\sigma = \sqrt{f^k(1 - f^k)n}$$

Sparsity coefficient:

$$S(C) = \frac{n(C) - f^k n}{\sqrt{f^k(1 - f^k)n}}$$

Interpretation:

$$S(C) < 0 \Rightarrow n(C) < E[n(C)] \Rightarrow \text{sparse cell}$$

$$S(C) \ll 0 \Rightarrow \text{objects in } C \text{ are likely outliers in that subspace}$$

Brute-force computational cost:

$$O(\varphi^k)$$

Angle-Based Outlier Factor – ABOF

For candidate point p and $x, y \in D \setminus \{p\}$:

$$\vec{px} = x - p, \quad \vec{py} = y - p$$

Pairwise angle-based quantity:

$$A_{x,y}(p) = \frac{\langle \vec{px}, \vec{py} \rangle}{\|\vec{px}\|^2 \|\vec{py}\|^2}$$

ABOF:

$$ABOF(p) = \text{VAR}_{x,y \in D} \left(\frac{\langle \vec{px}, \vec{py} \rangle}{\|\vec{px}\|^2 \|\vec{py}\|^2} \right)$$

Interpretation:

$$ABOF(p) \text{ small} \Rightarrow p \text{ is likely an outlier}$$

$$ABOF(p) \text{ large} \Rightarrow p \text{ is likely normal}$$

Computational cost:

$$O(n^3)$$

7 – XAI & Federated Learning

Feature Importance – Intuition

Given model score s , shuffle feature j K times and compute the corrupted scores $s(k, j)$:

$$I(j) = s - \frac{1}{K} \sum_{k=1}^K s(k, j)$$

Large decrease in performance $\Rightarrow$ feature j is important.

Constant feature:

$$I(j) = 0$$

SHAP – Shapley Values

Prediction decomposition:

$$f(x) = \mathbb{E}[f(X)] + \sum_{j=1}^J \phi_j$$

where

$$\mathbb{E}[f(X)] = \text{base value}$$

and ϕ_j is the contribution of feature j .

Shapley value:

$$\phi_j = \sum_{S \subseteq F \setminus \{j\}} \frac{|S|!(J - |S| - 1)!}{J!} [f(S \cup \{j\}) - f(S)]$$

Hence ϕ_j is the **average marginal contribution** of feature j over all possible coalitions.

$$|\phi_j| \uparrow \Rightarrow \text{feature impact} \uparrow$$

$$\phi_j > 0 \Rightarrow \text{positive contribution}$$

$$\phi_j < 0 \Rightarrow \text{negative contribution}$$

Federated Decision Trees

For regression trees:

$$\text{Var}(y) = \frac{1}{n} \sum_{i=1}^n (y_i - \bar{y})^2$$

Variance reduction for a split:

$$VR = \text{Var}(S) - \left[\frac{n_L}{n} \text{Var}(S_L) + \frac{n_R}{n} \text{Var}(S_R) \right]$$

Best split:

$$\text{split}^* = \arg \max_{\text{split}} VR$$

Useful federated form of the variance:

$$\text{Var}(y) = \frac{1}{n} \sum_{i=1}^n y_i^2 - \left(\frac{1}{n} \sum_{i=1}^n y_i \right)^2$$

Therefore each client only needs to send:

$$n_k, \quad \sum_i y_i, \quad \sum_i y_i^2$$

The server aggregates:

$$n = \sum_k n_k$$

$$\sum_i y_i = \sum_k \sum_{i \in P_k} y_i$$

$$\sum_i y_i^2 = \sum_k \sum_{i \in P_k} y_i^2$$

and computes the same variance that would be obtained centrally, without accessing raw data.

9 – Data Stream & Time Series

Damped Window Model

Older instances progressively lose importance:

$$f(t) = 2^{-\lambda t}$$

where λ is the decay rate:

$$\lambda \uparrow \Rightarrow \text{faster decay}$$

$$\lambda \downarrow \Rightarrow \text{older data remain relevant longer}$$

Computational Cost – Adaptive Streaming k-Means

Let:

$$L = \text{initial sequence length}, \quad d = \text{dimensions}$$

$$k = \text{clusters}, \quad cs = \text{number of centroid sets}$$

Estimate k for one dimension:

$$O(L)$$

Estimate k for all dimensions:

$$O(dL)$$

Run k-means for candidate centroid sets:

$$O(L d k cs)$$

Online assignment of a new instance:

$$O(k)$$

Total worst-case complexity:

$$O(dL) + O(L d k cs)$$

Computational Cost – MuDi-Stream

Let:

$$c = \text{number of core mini-clusters}$$

$$G = \text{total number of density grids}$$

Nearest core mini-cluster search:

$$O(c)$$

Map an instance to the grid:

$$O(\log \log G)$$

Pruning core mini-clusters:

$$O(c)$$

Pruning grids:

$$O(\log G)$$

Grid space complexity:

$$O(\log G)$$

Overall computational complexity:

$$O(c) + O(\log G)$$

Important:

complexity does not directly depend on N

where N is the total number of stream instances received.

VFDT – Hoeffding Bound

Let X have range R , with n observations.

$$\varepsilon = \sqrt{\frac{R^2 \ln(1/\delta)}{2n}}$$

where:

$$\delta = \text{probability of error}$$

$$1 - \delta = \text{confidence}$$

$$n \uparrow \Rightarrow \varepsilon \downarrow$$

For decision-tree splitting:

$$R = \ln c$$

where c is the number of classes.

Let X_A be the best attribute and X_B the second-best attribute:

$$\Delta \bar{G} = \bar{G}(X_A) - \bar{G}(X_B)$$

Split condition:

$$\Delta \bar{G} > \varepsilon$$

that is:

$$\bar{G}(X_A) - \bar{G}(X_B) > \sqrt{\frac{R^2 \ln(1/\delta)}{2n}}$$

If:

$$\Delta \bar{G} \leq \varepsilon \Rightarrow \text{wait for more examples}$$

If:

$$\Delta \bar{G} > \varepsilon \Rightarrow \text{split on } X_A$$

VFDT – Null Attribute / Pre-Pruning

The null attribute represents no split:

$$X_\emptyset = \text{do not split}$$

Its information gain is:

$$IG(X_\emptyset) = H(Y) - H(Y | X_\emptyset) = 0$$

Therefore the decision is:

$$\text{best attribute} \quad \text{vs.} \quad X_\emptyset$$

before selecting the best actual split.

VFDT – Expected Disagreement

Let:

$$p = P(\text{example reaching level } i \text{ falls into a leaf at level } i)$$

Then:

$$E[\text{disagreement}] \leq \frac{\delta}{p}$$

where disagreement is between the VFDT tree and the tree obtained with infinitely many examples.

$$p \uparrow \Rightarrow \frac{\delta}{p} \downarrow$$

$$p \downarrow \Rightarrow \frac{\delta}{p} \uparrow$$

VFDT – Memory Complexity

Sufficient statistics:

$$n_{ijk}$$

with:

$$i = \text{attribute}, \quad j = \text{attribute value}, \quad k = \text{class}$$

Memory:

$$O(\text{leaves} \times \text{attributes} \times \text{values} \times \text{classes})$$

10 – Time Series

MAPE

$$\text{MAPE} = \frac{100}{n} \sum_{i=1}^n \left| \frac{A_i - F_i}{A_i} \right|$$

A_i : actual value, F_i : forecast value.

Random Walk and Stationarity

General AR(1):

$$y_t = C + \alpha_1 y_{t-1} + \varepsilon_t$$

Stationarity condition:

$$-1 < \alpha_1 < 1$$

Random walk:

$$y_t = y_{t-1} + \varepsilon_t$$

First difference:

$$\Delta y_t = y_t - y_{t-1} = \varepsilon_t$$

Random walk:

$$\alpha_1 = 1 \Rightarrow \text{non-stationary}$$

Autocorrelation (ACF)

Autocorrelation at lag k :

$$r_k = \text{Corr}(y_t, y_{t-k}) = \frac{\sum_{t=k+1}^T (y_t - \bar{y})(y_{t-k} - \bar{y})}{\sum_{t=1}^T (y_t - \bar{y})^2}$$

$$r_1 = \text{Corr}(y_t, y_{t-1}), \quad r_2 = \text{Corr}(y_t, y_{t-2})$$

MA(q) identification:

$$\text{ACF cuts off after lag } q$$

Drift Method

Slope:

$$b = \frac{y_f - y_i}{N - 1}$$

Forecast:

$$\hat{y}_{N+h} = y_f + b h$$

where y_i, y_f are the first and last training observations.

Moving Average – MA(q)

$$y_t = \mu + \varepsilon_t + \sum_{i=1}^q \theta_i \varepsilon_{t-i}$$

Expanded:

$$y_t = \mu + \varepsilon_t + \theta_1 \varepsilon_{t-1} + \cdots + \theta_q \varepsilon_{t-q}$$

MA(2):

$$y_t = \mu + \varepsilon_t + \theta_1 \varepsilon_{t-1} + \theta_2 \varepsilon_{t-2}$$

$$q = \text{number of past error terms}$$

Autoregressive – AR(p)

$$y_t = C + \sum_{i=1}^p \phi_i y_{t-i} + \varepsilon_t$$

Expanded:

$$y_t = C + \phi_1 y_{t-1} + \cdots + \phi_p y_{t-p} + \varepsilon_t$$

AR(p) identification:

$$\text{PACF cuts off after lag } p$$

ARMA(p, q)

$$y_t = C + \sum_{i=1}^p \phi_i y_{t-i} + \sum_{j=1}^q \theta_j \varepsilon_{t-j} + \varepsilon_t$$

Special cases:

$$\text{ARMA}(p, 0) = \text{AR}(p)$$

$$\text{ARMA}(0, q) = \text{MA}(q)$$

AIC for model selection:

$$\text{AIC} = 2k - 2 \ln(\hat{L})$$

$$k = p + q, \quad \min \text{AIC}$$

ARIMA(p, d, q)

Ordinary differencing:

$$\Delta y_t = y_t - y_{t-1}$$

d :

$$d = \text{minimum number of differences needed for stationarity}$$

ARMA applied to the differenced series:

$$y'_t = C + \sum_{i=1}^p \phi_i y'_{t-i} + \sum_{j=1}^q \theta_j \varepsilon_{t-j} + \varepsilon_t$$

$$\text{ARIMA}(p, d, q)$$

p : AR order, d : integration order, q : MA order.

SARIMA

$$\text{SARIMA}(p, d, q)(P, D, Q)_m$$

Ordinary differencing:

$$\Delta y_t = y_t - y_{t-1}$$

Seasonal differencing:

$$\Delta_m y_t = y_t - y_{t-m}$$

With seasonal integration order D :

$$\Delta_m^D y_t = (1 - B^m)^D y_t$$

and ordinary integration order d :

$$\Delta^d y_t = (1 - B)^d y_t$$

where B is the backshift operator:

$$By_t = y_{t-1}, \quad B^m y_t = y_{t-m}$$

Seasonal AR terms:

$$P = 2 \Rightarrow y_{t-m}, y_{t-2m}$$

Seasonal MA terms:

$$Q = 2 \Rightarrow \varepsilon_{t-m}, \varepsilon_{t-2m}$$

For $m = 12$:

$$P = 2 \Rightarrow y_{t-12}, y_{t-24}$$

$$Q = 2 \Rightarrow \varepsilon_{t-12}, \varepsilon_{t-24}$$

$$\text{SARIMA}(p, d, q)(0, 0, 0)_m = \text{ARIMA}(p, d, q)$$

p = non-seasonal AR order
 d = ordinary differencing order
 q = non-seasonal MA order
 P = seasonal AR order
 D = seasonal differencing order
 Q = seasonal MA order
 m = observations per seasonal cycle

SARIMAX

$$\text{SARIMAX}(p, d, q)(P, D, Q)_m$$

SARIMAX adds exogenous variables to SARIMA:

$$y_t = \text{SARIMA component} + \beta^T \mathbf{x}_t$$

Expanded exogenous component:

$$\beta^T \mathbf{x}_t = \beta_1 x_{1,t} + \beta_2 x_{2,t} + \cdots + \beta_r x_{r,t}$$

Hence:

$$y_t = \text{SARIMA component} + \sum_{j=1}^r \beta_j x_{j,t}$$

Model hierarchy:

$$\text{SARIMAX} \xrightarrow{\text{no exogenous vars}} \text{SARIMA}$$

$$\text{SARIMAX} \xrightarrow{\text{no seasonality}} \text{ARIMAX}$$

$$\text{SARIMAX} \xrightarrow{\text{no seasonality, no exogenous vars}} \text{ARIMA}$$

Multi-step forecasting condition:

$$\hat{y}_{t+h} \text{ requires } \mathbf{x}_{t+h}$$